

Tópico 05: Electron + SQLite via IPC

UC2 – Banco de Dados Relacional

Prof. Jeronimo Silva

Roteiro da Aula

- 1. O Desafio da Segurança no Electron
- 2. A Analogia do Restaurante (O que é IPC?)
- 3. Passo 1: Comunicação IPC e Preload Scripts
- 4. Estrutura de Pastas Profissional
- 5. Passo 2: Consultando dados com SQLite
- 6. SQL Injection, Placeholders e try/catch
- 7. Passo 3: Gravando e Deletando dados



Checklist de Conceitos

- **Context Isolation:** Isolamento de contexto seguro para a tela.
- **IPC (Inter-Process Communication):** Comunicação entre processos.
- **ipcMain / ipcRenderer:** Canais de envio e recebimento do Electron.
- **Preload Script:** Ponte segura de comunicação (Garçom).
- **better-sqlite3:** Biblioteca síncrona/performática para SQLite.
- **SQL Injection & Placeholders:** Vulnerabilidade e blindagem de dados.

Do Navegador para o Desktop

No navegador tradicional (Chrome/Firefox), o JavaScript roda isolado em uma "sandbox".

- **Segurança:** O JS da web não pode ler ou gravar arquivos no seu HD.
- **O Híbridismo do Electron:** Combina o Chromium (exibição da tela) com o Node.js (acesso total ao sistema de arquivos e SO).
- **O Dilema:** Como acessar o banco de dados localmente sem deixar o aplicativo vulnerável a invasões?

A Regra de Ouro da Segurança

No Electron Moderno:

- `contextIsolation: true`
- `nodeIntegration: false`
- **O que significa:** O frontend (`renderer.js`) **NÃO** tem acesso a módulos do Node.js ou ao sistema de arquivos.
- **Como acessamos o banco então?** O frontend deve pedir para o processo principal (`main.js`) ler/escrever no banco de dados.



A Analogia do Restaurante (IPC)

Como o Electron funciona por baixo dos panos para ler arquivos/bancos:

- **Cliente (Renderer Process / Frontend):** Quer comer (pedir dados). Não entra na cozinha por higiene e segurança.
- **Garçom (IPC - Inter-Process Communication):** Leva o pedido do cliente até a cozinha e traz o prato de volta.
- **Cozinha (Main Process / Node.js):** Tem acesso às facas e ao fogo (sistema, HD, SQLite). Prepara o pedido de forma isolada.



1. 0 Cliente (Renderer Process)

O fluxo começa na tela (`renderer.js`) com a ação do usuário:

```
btnSomar.addEventListener('click', async () => {  
  const valA = Number(inputNumA.value);  
  const valB = Number(inputNumB.value);  
  
  // 1. Faz o pedido ao Garçon (api) e espera (await) a resposta  
  const resultado = await window.api.somarNumeros(valA, valB);  
  
  divResultadoSoma.textContent = resultado; // Exibe o resultado  
});
```

- `async` / `await` : Pedir dados leva tempo. Usamos `await` para esperar a resposta sem congelar a tela. A palavra `async` é obrigatória na função para permitir o `await`.
- `textContent` : Insere o resultado como texto puro na tela. É mais seguro que `innerHTML` porque evita injeções de código malicioso (XSS).

2. O Garçom (Preload Script)

Como o objeto `window.api` surgiu na tela? O `preload.js` cria essa ponte:

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('api', {
  somarNumeros: (a, b) => ipcRenderer.invoke('somar-numeros', { a, b })
});
```

- `'api'`: O nome do objeto criado no frontend (`window.api`). Pode ser qualquer nome livre por convenção.
- `somarNumeros`: A função exposta que o frontend chamou no slide anterior.

3. O Envelope (`ipcRenderer.invoke`)

O garçom precisa enviar o pedido de forma estruturada:

- `ipcRenderer.invoke('somar-numeros', ...)` : Envia uma mensagem assíncrona.
- `'somar-numeros'` : O nome do canal (ou envelope). É um texto livre!
- Como funciona: O "Garçom" anota os dados `(a, b)` em um envelope rotulado como `'somar-numeros'` e o arremessa para a cozinha.



4. A Cozinha (Main Process)

No arquivo `main.js`, a cozinha escuta o envelope e executa o processamento:

```
const { ipcMain } = require('electron');

// Escuta o canal 'somar-numeros' e processa o pedido
ipcMain.handle('somar-numeros', (event, { a, b }) => {
  return Number(a) + Number(b); // Devolve o prato pronto
});
```

- `ipcMain.handle('somar-numeros', ...)` : "Quando chegar um envelope marcado com `'somar-numeros'`, execute a soma e retorne o valor".
- `event` : Primeiro argumento. É injetado automaticamente pelo Electron contendo metadados da janela.
 - ⚠ Pegadinha de JS: É um evento de comunicação de rede (IPC), e NÃO o clique do mouse! O clique do mouse ficou para trás no navegador (DOM).
 - O dado real que você enviou (`{ a, b }`) entra sempre a partir do segundo parâmetro.

Setup do Laboratório Prático (Clone via VS Code)

Antes de iniciarmos a prática, vamos clonar o repositório pelo VS Code:

1. **Abra o VS Code:** Aperte `Ctrl + Shift + P` (ou `F1`) para abrir a Paleta de Comandos.
2. **Comando de Clone:** Digite `Git: Clone` e pressione `Enter`.
3. **Cole a URL do GitHub:** Cole o link do repositório:
`https://github.com/jeronimo-jrsilva/senac-573.git`
4. **Pasta de Destino:** Escolha a sua pasta pessoal de trabalho no Pendrive.
5. **Abrir Projeto:** Clique em `Open` quando o VS Code perguntar.
6. **Ajuste do `package.json`:** Na pasta `exercicios_electron`, confirme o caminho relativo:

```
"start": "..\\..\\..\\..\\electron-win32-x64\\electron.exe ."
```

Prática: Passo 1 (Comunicação IPC)

Vá para a pasta `exercicios_electron` e encontre as marcações:

- **Exemplo Resolvido 1:** Digite uma frase no input e veja a quantidade de caracteres calculada no Main Process.
- **Exercício 1 (Soma):** Capture os valores do input no `renderer.js`, envie para `window.api.somarNumeros(a, b)` e exiba a soma na tela.

Síncrono vs. Assíncrono no Electron

Como dividir os fluxos de tempo ao usar o banco local:

- **Acesso Síncrono (No Banco - `main.js`):** A gravação no arquivo `.db` local leva microssegundos. O `better-sqlite3` roda de forma síncrona para simplificar o backend, eliminando excesso de Promises.
- **Acesso Assíncrono (Na Comunicação IPC):** Atravessar a memória do sistema operacional entre os processos leva tempo. Por isso, na tela (`renderer.js`), as chamadas à API usam `await`.

Estrutura de Pastas Profissional

Para organizar o projeto e isolar o banco de dados:

```
meu-app/  
├── database/  
│   └── agenda.db  
├── src/  
│   ├── main.js  
│   ├── preload.js  
│   ├── renderer.js  
│   ├── index.html  
│   └── database/  
│       └── conexao.js  
└── package.json
```

← Banco físico (Fora da src)

← Código fonte

← Conexão modular

Conexão Modular (`src/database/conexao.js`)

A conexão é instanciada no processo principal, que tem acesso total ao disco:

```
const Database = require('better-sqlite3');
const path = require('path');

const dbPath = path.join(__dirname, '../../database/agenda.db');
const db = new Database(dbPath); // Abre ou cria o banco

db.prepare(`
  CREATE TABLE IF NOT EXISTS contatos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT, telefone TEXT, email TEXT UNIQUE
  )
`).run();

module.exports = db;
```



O Trio de Ouro do SQLite (.all, .get, .run)

Comandos para manipular os dados através do `better-sqlite3`:

- `.all(...)`
 - Retorna um array de objetos. Usado para `SELECT` gerais.
- `.get(...)`
 - Retorna a primeira linha encontrada. Usado para buscar por ID ou e-mail.
- `.run(...)`
 - Executa alterações no banco. Usado para `INSERT`, `UPDATE`, `DELETE`.



0 Processo Main Conectado (main.js)

No backend (`main.js`), importamos a conexão para usar nas rotas IPC:

```
// 1. IMPORTA A CONEXÃO MODULAR
const db = require('./database/conexao');

// 2. Executa a query dentro do Handler IPC
ipcMain.handle('buscar-por-email', (event, email) => {
  const stmt = db.prepare('SELECT * FROM contatos WHERE email = ?');
  return stmt.get(email); // Devolve o contato encontrado
});
```

- `require('./database/conexao')` : Conecta a "Cozinha" ao banco de dados no disco físico.
- `db.prepare(...)` : Prepara a instrução SQL de forma síncrona.
- `stmt.get(...)` : Retorna os dados usando o método do Trio de Ouro.

Prática: Passo 2 (Consultar SQLite)

Vá para a pasta `exercicios_electron` e encontre as marcações:

- **Exemplo Resolvido 2:** Clique em "Carregar Todos" para ler os contatos no banco usando `.all()`.
- **Exercício 2 (Buscar por E-mail):** Obtenha o e-mail inserido e acione `window.api.buscarPorEmail(email)` (que executa `.get()`) para exibir o contato achado.

O Perigo da Injeção de SQL

Por que **NUNCA** devemos concatenar strings digitadas na query SQL:

```
// CÓDIGO VULNERÁVEL (NUNCA FAÇA ISSO):  
const query = "SELECT * FROM contatos WHERE nome = '" + input + "'";
```

- Se o usuário digitar no input de busca: `' OR 1=1; --`
- A query final se torna:
`SELECT * FROM contatos WHERE nome = ' ' OR 1=1; --'`
- O banco retornará **todos** os contatos da base, expondo dados sigilosos.

0 Escudo dos Placeholders (?)

Parametrizar os dados impede que o motor do banco interprete strings digitadas como comandos.

```
// CÓDIGO BLINDADO (RECOMENDADO):  
const stmt = db.prepare("SELECT * FROM contatos WHERE nome = ?");  
const resultado = stmt.all(input);
```

- O `better-sqlite3` escapa o input `' OR 1=1; --'`.
- Ele busca literalmente por alguém chamado `' OR 1=1; --'`, sem executar comandos.

Prevenção de Travamentos (try/catch)

Erros de restrição (ex: cadastrar e-mail duplicado) quebram o Main Process se não tratados:

```
ipcMain.handle('inserir-contato', (event, { nome, email }) => {  
  try {  
    const stmt = db.prepare('INSERT INTO contatos ... VALUES (?, ?)');  
    stmt.run(nome, email);  
    return { success: true };  
  } catch (error) {  
    return { success: false, error: error.message };  
  }  
});
```



Como Funciona o try/catch?

A estrutura de tratamento de exceções no Javascript:

- **try** (Tente): O bloco de código que tentamos executar. Se nada falhar, o bloco **catch** é ignorado.
- **catch (error)** (Capture): Se ocorrer qualquer erro/crash dentro do **try** (ex: e-mail duplicado), o JS interrompe a execução na hora e desvia para o **catch** com o detalhe do erro.
- **Diferença do if/else** : O **if/else** testa condições lógicas. O **try/catch** captura falhas físicas e erros graves que derrubariam o aplicativo.

Prática: Passo 3 (Alterações no SQLite)

Vá para a pasta `exercicios_electron` e encontre as marcações:

- **Exemplo Resolvido 3:** Formulário de cadastro de contatos enviando dados de forma segura com placeholders e tratando erros.
- **Exercício 3 (Excluir por ID):** Leia o ID inserido no campo, chame a API `window.api.deletarContato(id)` e exiba a mensagem de sucesso se a deleção afetar alguma linha.

Prática: Passo 4 & 5 (Avançado)

Exercícios finais para consolidar a integração:

- **Exercício 4 (Busca Dinâmica com LIKE)**: Use o evento de digitação `'input'` para disparar `window.api.buscarPorNome(texto)` e recarregar a lista dinamicamente conforme o usuário digita.
- **O Evento `'input'`**: Fired (disparado) imediatamente sempre que o texto do campo muda (digitar, colar ou deletar). Ideal para buscas em tempo real.
- **Exercício 5 (Estatísticas com COUNT)**: Obtenha a quantidade total de contatos no banco com `SELECT COUNT(*)` e atualize o contador na tela.

🚩 Conclusão & Próximos Passos

Você aprendeu a conectar e manipular bancos de dados locais embutidos em aplicações Desktop híbridas de forma segura utilizando a arquitetura IPC do Electron.

Na próxima aula (Tópico 06):

- Iniciaremos a modelagem conceitual de dados utilizando Diagramas Entidade-Relacionamento (DER).
- Entenderemos como ligar múltiplas tabelas através de Chaves Estrangeiras.